COMP/EECE 7/8740 Neural Networks

Assignment # 2
Spring 2026
Due: March 4, 2026
Total points: 100

This assignment aims to designing a new deep learning architecture and evaluating Transfer Learning (TL) and ensemble methods to achieve better performance, generalization ability and robustness of the models.

Task 1 (Points: 50):

The objective of the first task is to develop a new Deep Convolutional Neural Networks (DCNN) architecture from scratch and validating the models with **Tiny ImageNet-200 (200 different types)** classification datasets.

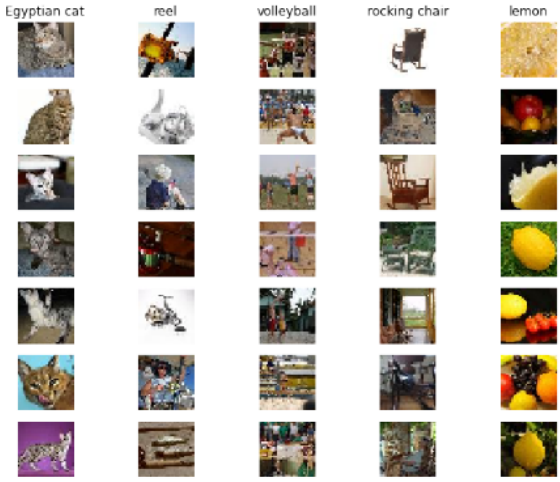

Figure 1. Example images from Tiny ImageNet-200 datasets.

Implementation Requirements

Dataset and preprocessing steps:

- Input image size: **64 × 64 × 3 (**resize input images to **32 × 32 × 3)** in order to be consistent with respect to CIFAR-100 for transfer learning.
- Normalize both training and testing image sets

DCNN Architecture Designing and Optimization:

- **DCNN Architecture**:
    - Use an AI-assistive system to design a novel deep convolutional neural network architecture (reference architectures such as ResNet, DenseNet, FractralNet etc.)

    - Also consider network parameters optimization methods:
        - The alternative of fully connected layers such as the Global average pooling (GAP) layer
        - Utilize 1x1 convolution instead of fully connected layers.
    - Regularization with dropout and
    - Batch normalization

- **Activation Function:** along with activation functions such as ReLU, and GELU, report the performance of the new network using custom activation function developed in Assignment_1 (Task 2).
- **Model Optimization**
    - Train the model using the following optimization methods:
        - Adam Weighted (AdamW) optimizer

Evaluation and finalization: Design a novel architecture that achieves superior performance compared to baseline models such as ResNet-18/50 and DenseNet (depending on the availability of computational resources, use Google Colab).

After designing and validating the architectures, apply transfer learning (TL) to achieve improved performance on the CIFAR-100 object classification task:

- Train and evaluate the model on CIFAR-100 using randomly initialized weights (or He initialization).
- Select the best-performing model, initialize it with pretrained weights from Tiny ImageNet-200, and fine-tune it on the CIFAR-100 dataset.

Training Analysis

- Plot **training and validation loss curves** to demonstrate learning behavior and convergence during training for finetuning the model.

Model Evaluation

- Evaluate the trained model on the test dataset (CIFAR-100) using the following performance metrics:
    - Accuracy
    - Precision
    - Recall
    - F1-score
    - Confusion matrix
    - ROC curve
    - Precision–Recall curve

Expected Outcome

- Interpret the results using quantitative metrics and visualizations for CIFAR-100
- Present the **overall performance** of the model **without applying any thresholding**.
- Additionally, **compute and report performance metrics for predictions with confidence scores greater than 0.9**, treating this value as an **industrial-level decision threshold** for the task.

Notes:

1. Data Splits (already processed):
    a. **Training Set:** 100,000 images total (500 images per class).

    b. **Test Set:** 10,000 images total (50 images per class)

2. Training samples (if you think the number of samples are too much to train, use 100/200 per class from training sets): consider randomly selected 80% of the samples from each class for training and the remaining 20% for validation the model, respectively.
3. Testing samples: use the independent testing example for evaluation the model performance
4. For report writing for task_1, you are allowed to use the mathematical intuition of the new DCNN architecture generated by AI assistive system.

Task 2 (Points: 50):

The objective of the second task is to implement and evaluate ensemble models using the best model from Task_1 to achieve further improved accuracy for the **CIFAR-100 object (100 different types)** classification task.

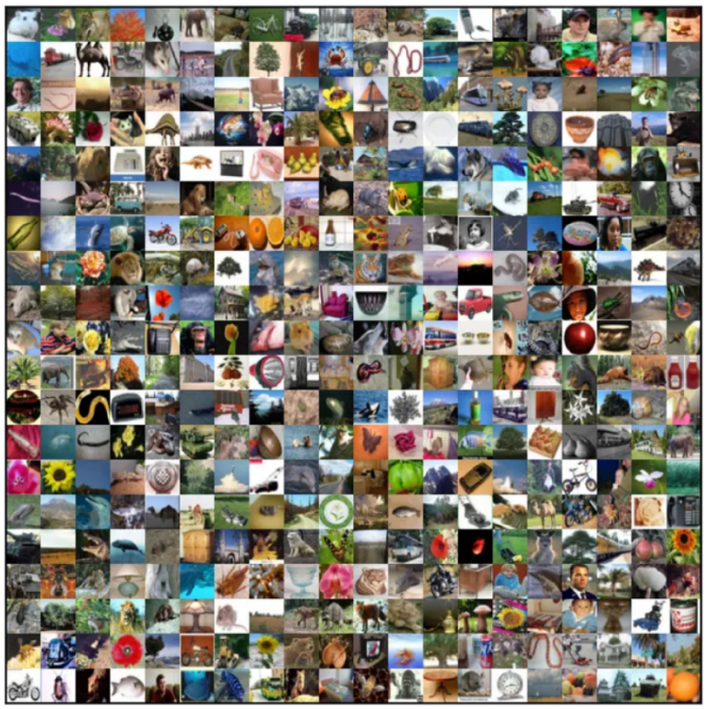

Figure 2. Example images from CIFAR100 dataset.

Implementation Steps:

Data Preprocessing

- Image size: 32x32x3
- Normalize both training and testing image sets

Ensemble method:

Perform the empirical evaluation to select the best model in Task_1 and then **ensemble the best model considering odd number of models (3)** and revaluated the performance CIFAR-100 object recognition tasks.

Training Analysis

- Plot **training and validation loss curves** to demonstrate learning behavior and convergence during training

Model Evaluation

- Evaluate the trained model on the test dataset using the following performance metrics:
  - Accuracy
  - Precision
  - Recall
  - F1-score
  - Confusion matrix
  - ROC curve
  - Precision–Recall curve

Expected Outcome

- Interpret the results using quantitative metrics and visualizations
- Present the **overall performance** of the model **without applying any thresholding**.
- Additionally, **compute and report performance metrics for predictions with confidence scores greater than 0.9**, treating this value as an **industrial-level decision threshold** for the task.

Compare the results obtained without transfer learning, with transfer learning, and using ensemble methods, and report the findings accordingly.

Datasets in CANVAS:

- Task 1 : https://www.kaggle.com/datasets/akash2sharma/tiny-imagenet
- Task 2: torchvision.datasets.CIFAR100

Ref. codes available in CANVAS:

- Task 1: ~/CANVAS/Files/Ref.codes/Lecture_6/7/8_*.py
- Task 2: ~/CANVAS/Files/Ref.codes/Lecture_7/8_ *.py

Final Note: You must submit **fully executable source code** for **both tasks** as part of this assignment.

Report Outline

1. Title Page
- Course Title
- Course Number
- Student Name
- Student ID
- Assignment Number
- Submission Date (Month, Year)

2. Abstract

(Starts on the second page)
- Brief problem statement
- Dataset and methodology overview
- Key results and performance highlights
- Main conclusions

3. Introduction

3.1 Problem Definition
- Description of the classification/regression task
- Motivation and real-world relevance

3.2 Background and Related Work
- Overview of CNN/DNN-based approaches
- Summary of relevant prior studies and models

3.3 Objectives and Contributions
- Clear project objectives
- Novel aspects (e.g., custom architecture, new activation function, optimizer comparison)
- Summary of technical contributions

4. Methodology
- Overall system pipeline
- Data preprocessing steps
- Training and testing split
- Evaluation strategy

5. Neural Network Model Descriptions
- DNN/DCNN architecture design for regression and classification
- Layer-wise model configuration
- Activation functions (standard vs. proposed)
- Optimization techniques (SGD + momentum, Adam)
- Loss functions and hyperparameters

6. Experiments and Results

6.1 Dataset Description

- Dataset source and structure
- Number of classes and samples
- Input dimensions

6.2 Training and Testing Performance

- Training and validation loss curves
- Accuracy and convergence behavior

6.3 Quantitative Evaluation Metrics

- Accuracy, Precision, Recall, F1-score
- ROC and Precision–Recall curves

6.4 Comparative Analysis

- Performance comparison across:
  - Activation functions
  - Optimizers
- Discussion of observed trends

6.5 Limitations and Challenges

- Computational constraints
- Dataset limitations
- Model generalization issues

7. Conclusion and Future Work

- Summary of findings
- Key takeaways
- Suggestions for future improvements or extensions

8. References

- Properly formatted academic references (APA / IEEE)

9. Appendix (Optional)

- Network hyperparameters
- Additional plots
- Pseudocode or algorithm descriptions